Offline Verification of kNN Prediction Override for Highly Frequent Drugs on Hybrid OOFs

Question. The test set contains (at least) five highly frequent drugs from train at the same frequency. If a test row is almost a duplicate of a row belonging to one of these drugs, is it beneficial to pull its prediction towards the label profile of that drug?

What the notebook does (no GPU, no retraining; uses OOFs from hybrid run H2):

1. **Configuration and utilities:** Define hyper-parameters, load environment paths, set evaluation thresholds, and implement utility functions for directory search and row-level log loss computation.
2. **OOF data loading and feature alignment:** Load hybrid model OOF predictions, targets, and drug mappings, filter out control samples (`ctl_vehicle`), identify frequent target drugs, and calculate the baseline blend OOF loss and theoretical gain ceiling.
3. **Fold-level kNN representation, rules, and override variants:** Compute fold-specific embeddings, estimate distance thresholds (`thr_X`), label profiles, and assignment precisions on training splits, then generate override prediction variants (`hard`, `mix_precision`, and fixed-weight mixtures) for validation rows.
4. **Evaluation of OOF loss impact:** Measure row-level log loss changes for each override variant, evaluate performance on true assignments vs. false positives, and compute fold-by-fold OOF loss deltas.
5. **Per-drug breakdown and precision analysis:** Identify the best-performing override variant, quantify per-drug contributions to the OOF loss change, and inspect estimated rule precisions across folds.
6. **Decision criteria and expected test gain translation:** Check adoption thresholds against OOF performance metrics, translate loss changes per assigned row into an estimated test leaderboard gain, and output the final decision and caveats[cite: 2].

Why it matters: The gain ceiling is the current loss on frequent rows (~0.002 per row, ~7% of test). The notebook measures how much is recoverable *and* how much misassignments cost[cite: 2].

Input: Run H2 folder with `oof_resnet.npy`, `oof_ft_transformer.npy`, `oof_catboost.npy`, `oof_targets.npy`, `cv_fold_of_row.npy`, `train_drug_ids.npy`, `metadata.json` (and `train_sig_ids.npy` if present), plus competition data (`train_features.csv`, `train_targets_scored.csv`, `train_drug.csv`)[cite: 2].

In [1]:
```
# Step 1 - configuration and utilities
import os, glob, json, time
import numpy as np
import pandas as pd
```

```
RUN_DIR = "/kaggle/input/datasets/marcomerlo37/moa-knn-dataset"
DATA_DIR = None                  # None: search train_features.csv under /kaggle/input
FREQUENT_MIN_ROWS = 18           # same as HYBRID_MIN_ROWS
MIN_DRUG_ROWS = 20               # frequent drugs used by rule (as in test analysis: 8 drugs)
THRESHOLD_QUANTILE = 0.90        # threshold thr_X = 90th percentile of distance for actual drug X rows
PROFILE_SMOOTH = 0.5             # label profile smoothing: (positives + a) / (n + 2a)
FIXED_LAMBDAS = [0.5, 0.9]       # fixed-weight mixtures
MIN_GAIN = 3e-5                  # threshold on OOF loss gain (average over all rows)
MIN_FOLDS_IMPROVED = 4
TEST_ASSIGNED_RATE = 334 / 3624  # test rows assigned to the 8 drugs (test analysis)
TEST_TREATED_SHARE = 0.910
CLIP_EPS = 1e-15
CHUNK = 1000
CTL_LABEL = "ctl_vehicle"

def find_input_directory(file_name):
    matches = glob.glob(f"/kaggle/input/**/{file_name}", recursive=True)
    directories = sorted({os.path.dirname(p) for p in matches})
    if len(directories) != 1:
        raise FileNotFoundError(f"Expected a single folder with '{file_name}' under /kaggle/input, found {len(directories)}: {directories}")
    return directories[0]

if DATA_DIR is None:
    DATA_DIR = find_input_directory("train_features.csv")
drug_path = os.path.join(DATA_DIR, "train_drug.csv")
if not os.path.exists(drug_path):
    drug_path = os.path.join(find_input_directory("train_drug.csv"), "train_drug.csv")

def loss_rows(Y, P, eps=CLIP_EPS):
    """Mean log loss across cells for each row, shape (rows,)."""
    P = np.clip(np.asarray(P, dtype=np.float64), eps, 1 - eps)
    return (-(Y * np.log(P) + (1 - Y) * np.log(1 - P))).mean(axis=1)
print("Run:", RUN_DIR, "| data:", DATA_DIR)
```

```
Run: /kaggle/input/datasets/marcomerlo37/moa-knn-dataset | data: /kaggle/input/competitions/lish-moa
```

In [2]:

```
# Step 2 - load OOFs and realign features to OOF rows
with open(f"{RUN_DIR}/metadata.json") as f:
    metadata = json.load(f)
weights, target_cols = metadata["weights"], metadata["target_cols"]
print("Run CV scheme:", metadata.get("cv_scheme", "unknown"), "| blend weights:", {k: round(v, 4) for k, v in weights.items()})
Y = np.load(f"{RUN_DIR}/oof_targets.npy")
fold_of_row = np.load(f"{RUN_DIR}/cv_fold_of_row.npy")
saved_drugs = np.load(f"{RUN_DIR}/train_drug_ids.npy", allow_pickle=True).astype(str)
blend = sum(weights[name] * np.load(f"{RUN_DIR}/oof_{name}.npy") for name in weights)
n_total, n_targets = Y.shape
N_FOLDS = int(fold_of_row.max()) + 1
```

```
train = pd.read_csv(f"{DATA_DIR}/train_features.csv").sort_values("sig_id").reset_index(drop=True)
targets = pd.read_csv(f"{DATA_DIR}/train_targets_scored.csv").sort_values("sig_id").reset_index(drop=True)
assert (train["sig_id"].values == targets["sig_id"].values).all()
treated = (train["cp_type"] != CTL_LABEL).values
train, targets = train[treated].reset_index(drop=True), targets[treated].reset_index(drop=True)
assert len(train) == n_total, f"different rows: features {len(train)} vs OOF {n_total}"
assert (targets[target_cols].values == Y).all(), "labels do not match oof_targets: different row order"
sig_path = f"{RUN_DIR}/train_sig_ids.npy"
if os.path.exists(sig_path):
    assert (np.load(sig_path, allow_pickle=True).astype(str) == train["sig_id"].values.astype(str)).all(), "different sig_ids"
drug_map = pd.read_csv(drug_path).set_index("sig_id")["drug_id"]
drug_ids = train["sig_id"].map(drug_map).astype(str).values
assert (drug_ids == saved_drugs).all(), "drugs do not match train_drug_ids.npy"

g_cols = [c for c in train.columns if c.startswith("g-")]
c_cols = [c for c in train.columns if c.startswith("c-")]
G, C = train[g_cols].values.astype(np.float32), train[c_cols].values.astype(np.float32)
codes, drug_names = pd.factorize(drug_ids)
rows_per_drug = np.bincount(codes)
rule_drugs = [int(x) for x in np.where(rows_per_drug >= MIN_DRUG_ROWS)[0] if rows_per_drug[x] > FREQUENT_MIN_ROWS]
before = loss_rows(Y, blend)
print(f"Rows {n_total} | targets {n_targets} | folds {N_FOLDS} | blend OOF loss {before.mean():.5f}")
print(f"Frequent drugs used by rule: {len(rule_drugs)} ({rows_per_drug[rule_drugs].sum() / n_total:.1%} of rows)")

# are labels truly per-drug? (if so, X's profile describes each of its rows well)
rows = []
for X in rule_drugs:
    idx = np.where(codes == X)[0]
    modal_share = pd.Series([tuple(r) for r in Y[idx].astype(int)]).value_counts(normalize=True).iloc[0]
    rows.append({"drug": drug_names[X], "rows": len(idx), "modal_label_share": modal_share,
                 "current_blend_loss": before[idx].mean(), "positive_targets": int(Y[idx].max(axis=0).sum())})
purity = pd.DataFrame(rows)
print(purity.round(4).to_string(index=False))
ceiling = before[np.isin(codes, rule_drugs)].sum() / n_total
print(f"\nGain ceiling: current loss on rows of these drugs / total rows = {ceiling:.2e} (if loss on them went to 0)")
```

```
Run CV scheme: hybrid | blend weights: {'resnet': 0.478, 'ft_transformer': 0.3475, 'catboost': 0.1745}
Rows 21948 | targets 206 | folds 5 | blend OOF loss 0.01715
Frequent drugs used by rule: 8 (9.7% of rows)
     drug  rows  modal_label_share  current_blend_loss  positive_targets
8b87a7a83   203               1.0              0.0009                 1
87d714366   718               1.0              0.0006                 2
292ab2c28   194               1.0              0.0021                 0
5628cb3ee   202               1.0              0.0017                 1
d50f18348   186               1.0              0.0059                 3
9f80f3f77   246               1.0              0.0031                 1
d1b47f29d   178               1.0              0.0009                 1
d08af5d4b   196               1.0              0.0016                 1

Gain ceiling: current loss on rows of these drugs / total rows = 1.68e-04 (if loss on them went to 0)
```

In [3]:

```python
# Step 3 - nearest neighbor, assignment rule, profiles, and override variants
def nearest(Q, P, same=False):
    """Cosine distance (1 - similarity) from nearest neighbor and index; same=True excludes self-match (when Q and P coincide)."""
    d1 = np.empty(len(Q), dtype=np.float32); arg = np.empty(len(Q), dtype=np.int64)
    for s in range(0, len(Q), CHUNK):
        sim = Q[s:s + CHUNK] @ P.T
        if same:
            sim[np.arange(sim.shape[0]), s + np.arange(sim.shape[0])] = -np.inf
        a = sim.argmax(axis=1)
        arg[s:s + CHUNK] = a
        d1[s:s + CHUNK] = 1.0 - sim[np.arange(len(a)), a]
    return d1, arg

def embed(idx, stats):
    """As in test analysis: standardized g- and c- blocks (fold training statistics only), L2 normalized, concatenated."""
    blocks = []
    for name, M in (("g", G), ("c", C)):
        mu, sd = stats[name]
        z = (M[idx] - mu) / sd
        blocks.append(z / (np.linalg.norm(z, axis=1, keepdims=True) + 1e-12))
    return (np.hstack(blocks) / np.sqrt(2)).astype(np.float32)

def assign(d1, nn_code, thr):
    out = np.full(len(d1), -1, dtype=int)
    for X, t in thr.items():
        out[(nn_code == X) & (d1 <= t)] = X
    return out

VARIANTS = ["hard", "mix_precision"] + [f"mix_{lam}" for lam in FIXED_LAMBDAS]
pred_after = {v: blend.copy() for v in VARIANTS}
assigned_drug = np.full(n_total, -1, dtype=int)
fold_params = []
t0 = time.time()
```

```
for f in range(N_FOLDS):
    V = np.where(fold_of_row == f)[0]; T = np.where(fold_of_row != f)[0]
    stats = {"g": (G[T].mean(0), G[T].std(0) + 1e-8), "c": (C[T].mean(0), C[T].std(0) + 1e-8)}
    XT, XV = embed(T, stats), embed(V, stats)
    codes_T = codes[T]

    # parameters estimated ONLY on fold training split (leave-one-out nearest neighbor)
    d_T, arg_T = nearest(XT, XT, same=True)
    nn_code_T = codes_T[arg_T]
    thr, profile, precision = {}, {}, {}
    for X in rule_drugs:
        of_X = codes_T == X
        if of_X.sum() < 10:
            continue
        thr[X] = float(np.quantile(d_T[of_X], THRESHOLD_QUANTILE))
        positives = Y[T][of_X].sum(axis=0)
        profile[X] = (positives + PROFILE_SMOOTH) / (of_X.sum() + 2 * PROFILE_SMOOTH)
    assigned_T = assign(d_T, nn_code_T, thr)
    for X in thr:
        picked = assigned_T == X
        precision[X] = float((codes_T[picked] == X).mean()) if picked.any() else 0.0

    # validation: assignment and override
    d_V, arg_V = nearest(XV, XT)
    assigned_V = assign(d_V, codes_T[arg_V], thr)
    assigned_drug[V] = assigned_V
    for X in thr:
        rows = V[assigned_V == X]
        if len(rows) == 0:
            continue
        pred_after["hard"][rows] = profile[X]
        pred_after["mix_precision"][rows] = (min(precision[X], 0.99) * profile[X]
                                             + (1 - min(precision[X], 0.99)) * blend[rows])
        for lam in FIXED_LAMBDAS:
            pred_after[f"mix_{lam}"][rows] = lam * profile[X] + (1 - lam) * blend[rows]
    fold_params.append({"fold": f, "precision_training": {drug_names[X]: round(p, 3) for X, p in precision.items()}})
    print(f"fold {f}: rows assigned in validation {int((assigned_V >= 0).sum())} | {time.time() - t0:.0f}s")
```

```
fold 0: rows assigned in validation 474 | 6s
fold 1: rows assigned in validation 479 | 12s
fold 2: rows assigned in validation 473 | 18s
fold 3: rows assigned in validation 496 | 25s
fold 4: rows assigned in validation 459 | 31s
```

In [4]:

```
# Step 4 - effect on OOF loss
after = {v: loss_rows(Y, pred_after[v]) for v in VARIANTS}
is_assigned = assigned_drug >= 0
is_true = is_assigned & (codes == assigned_drug)          # assigned and truly belonging to that drug
```

```
is_false = is_assigned & ~is_true                          # false positive (different drug)
print(f"Assigned rows: {int(is_assigned.sum())} ({is_assigned.mean():.1%} of train) | correct {int(is_true.sum())} | "
      f"incorrect {int(is_false.sum())} | validation precision {is_true.sum() / max(1, is_assigned.sum()):.3f}")
in_rule = np.isin(codes, rule_drugs)
print(f"Actual frequent drug rows: {int(in_rule.sum())} | assigned to correct drug (recall): {is_true.sum() / in_rule.sum():.3f}")

rows = []
for v in VARIANTS:
    delta_rows = after[v] - before
    per_fold = [delta_rows[(fold_of_row == f) & is_assigned].sum() / (fold_of_row == f).sum() for f in range(N_FOLDS)]
    rows.append({"variant": v,
                 "delta_loss_OOF": delta_rows[is_assigned].sum() / n_total,
                 "on_correct_rows": delta_rows[is_true].sum() / n_total,
                 "on_false_positives": delta_rows[is_false].sum() / n_total,
                 "assigned_loss_before": before[is_assigned].mean(), "assigned_loss_after": after[v][is_assigned].mean(),
                 "folds_improved": int(sum(d < 0 for d in per_fold)),
                 "delta_per_fold(1e-5)": np.round(np.array(per_fold) * 1e5, 1).tolist()})
variants_report = pd.DataFrame(rows)
pd.set_option("display.width", 250)
print("\ndelta_loss_OOF: mean loss change across ALL rows (negative = better). Ceiling:", f"{-ceiling:.2e}")
print(variants_report.round(6).to_string(index=False))
```

```
Assigned rows: 2381 (10.8% of train) | correct 1718 | incorrect 663 | validation precision 0.722
Actual frequent drug rows: 2123 | assigned to correct drug (recall): 0.809

delta_loss_OOF: mean loss change across ALL rows (negative = better). Ceiling: -1.68e-04
      variant  delta_loss_OOF  on_correct_rows  on_false_positives  assigned_loss_before  assigned_loss_after  folds_improved
delta_per_fold(1e-5)
         hard        0.001469         0.000084            0.001385              0.005822             0.019361               0 [139.3, 134.3, 155.5,
178.4, 127.1]
mix_precision        0.000258         0.000070            0.000188              0.005822             0.008197               0      [21.2, 27.7, 24.1,
30.0, 25.8]
      mix_0.5        0.000209         0.000040            0.000169              0.005822             0.007749               0      [18.7, 20.9, 21.4,
26.0, 17.6]
      mix_0.9        0.000643         0.000075            0.000568              0.005822             0.011750               0      [59.7, 61.6, 67.2,
78.2, 55.0]
```

In [5]:

```
# Step 5 - per-drug breakdown (best variant) and per-fold precision
best = variants_report.loc[variants_report["delta_loss_OOF"].idxmin(), "variant"]
print(f"Best variant for delta_loss_OOF: {best}\n")
rows = []
for X in rule_drugs:
    picked = assigned_drug == X
    if not picked.any():
        continue
    own = picked & (codes == X)
    rows.append({"drug": drug_names[X], "train_rows": int(rows_per_drug[X]), "assigned": int(picked.sum()),
                 "precision": own.sum() / picked.sum(),
```

```
                "loss_before": before[picked].mean(), "loss_after": after[best][picked].mean(),
                "delta_OOF_contribution": (after[best][picked] - before[picked]).sum() / n_total})
per_drug = pd.DataFrame(rows).sort_values("delta_OOF_contribution")
print(per_drug.round(5).to_string(index=False))
print("\nRule precision estimated on training split of each fold (used by mix_precision):")
for fp in fold_params:
    print(f"  fold {fp['fold']}: {fp['precision_training']}")
```

```
Best variant for delta_loss_OOF: mix_0.5

     drug  train_rows  assigned  precision  loss_before  loss_after  delta_OOF_contribution
8b87a7a83         203       187    0.97326      0.00087     0.00197                 0.00001
d1b47f29d         178       160    0.97500      0.00094     0.00222                 0.00001
87d714366         718       684    0.95175      0.00237     0.00277                 0.00001
292ab2c28         194       208    0.77885      0.00361     0.00507                 0.00001
d08af5d4b         196       242    0.66942      0.00540     0.00668                 0.00001
5628cb3ee         202       216    0.65278      0.00712     0.00902                 0.00002
9f80f3f77         246       395    0.37468      0.01221     0.01513                 0.00005
d50f18348         186       289    0.40138      0.01215     0.01813                 0.00008

Rule precision estimated on training split of each fold (used by mix_precision):
  fold 0: {'8b87a7a83': 0.98, '87d714366': 0.95, '292ab2c28': 0.828, '5628cb3ee': 0.612, 'd50f18348': 0.347, '9f80f3f77': 0.342, 'd1b47f29d': 0.985,
'd08af5d4b': 0.696}
  fold 1: {'8b87a7a83': 1.0, '87d714366': 0.948, '292ab2c28': 0.81, '5628cb3ee': 0.675, 'd50f18348': 0.38, '9f80f3f77': 0.399, 'd1b47f29d': 0.984,
'd08af5d4b': 0.717}
  fold 2: {'8b87a7a83': 0.98, '87d714366': 0.948, '292ab2c28': 0.804, '5628cb3ee': 0.653, 'd50f18348': 0.367, '9f80f3f77': 0.385, 'd1b47f29d': 0.985,
'd08af5d4b': 0.681}
  fold 3: {'8b87a7a83': 0.98, '87d714366': 0.948, '292ab2c28': 0.839, '5628cb3ee': 0.655, 'd50f18348': 0.376, '9f80f3f77': 0.45, 'd1b47f29d': 0.977,
'd08af5d4b': 0.733}
  fold 4: {'8b87a7a83': 0.98, '87d714366': 0.954, '292ab2c28': 0.818, '5628cb3ee': 0.765, 'd50f18348': 0.458, '9f80f3f77': 0.389, 'd1b47f29d': 0.992,
'd08af5d4b': 0.744}
```

In [6]:

```
# Step 6 - decision and translation into expected test gain
best_row = variants_report.set_index("variant").loc[best]
gain = -best_row["delta_loss_OOF"]
adoptable = gain >= MIN_GAIN and best_row["folds_improved"] >= MIN_FOLDS_IMPROVED
per_assigned_row = best_row["delta_loss_OOF"] * n_total / is_assigned.sum()          # mean change per assigned row
test_gain = -per_assigned_row * TEST_ASSIGNED_RATE * TEST_TREATED_SHARE              # leaderboard all-rows scale
print(f"Variant {best}: OOF loss gain {gain:.2e} ({gain / before.mean():.2%} of loss) | "
      f"folds improved {int(best_row['folds_improved'])}/{N_FOLDS}")
print(f"Gain per assigned row: {-per_assigned_row:.4f} | expected test gain (all-rows scale): ~{test_gain:.1e} "
      f"(assuming {TEST_ASSIGNED_RATE:.1%} of test rows assigned)")
print(f"Adoption threshold: gain >= {MIN_GAIN:.0e} and >= {MIN_FOLDS_IMPROVED}/{N_FOLDS} folds improved -> ", "ADOPTABLE" if adoptable else "NOT
adoptable")
print("\nCaveats: (1) validation uses 80% of train as pool, test will use 100%; "
      "(2) test frequent rows could originate from different plates than train; "
      "(3) a gain of order 1e-5..1e-4 on the leaderboard is indistinguishable from noise: verification is offline only, on OOFs.")
```

Variant mix_0.5: OOF loss gain -2.09e-04 (-1.22% of loss) | folds improved 0/5
Gain per assigned row: -0.0019 | expected test gain (all-rows scale): ~-1.6e-04 (assuming 9.2% of test rows assigned)
Adoption threshold: gain >= 3e-05 and >= 4/5 folds improved -> NOT adoptable

Caveats: (1) validation uses 80% of train as pool, test will use 100%; (2) test frequent rows could originate from different plates than train; (3) a gain of order 1e-5..1e-4 on the leaderboard is indistinguishable from noise: verification is offline only, on OOFs.